

104. 二叉树的最大深度

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Easy	(Not Available)	(Not Available)	-	-	0

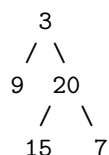
- **Tags:** 树, 深度优先搜索, 广度优先搜索, 二叉树
- **Companies:** (Not Available)

给定一个二叉树，找出其最大深度。

二叉树的深度为根节点到最远叶子节点的最长路径上的节点数。

说明: 叶子节点是指没有子节点的节点。

示例: 给定二叉树 [3,9,20,null,null,15,7],



返回它的最大深度 3。

提示:

- 树中节点的数量在 $[0, 10^4]$ 区间内。
- $-100 \leq \text{Node.val} \leq 100$

[Discussion](#) | [Solution](#)

解决方法

1. 问题理解

目标: 计算给定二叉树的最大深度。**定义:** 最大深度是指从根节点到最远的叶子节点所经过的节点数量。**关键点:** - 空树的深度为 0。- 只有一个根节点的树深度为 1。- 叶子节点是没有子节点的节点。

2. 思路分析

方法一: 递归 (深度优先搜索 - DFS) 树的深度问题天然适合使用递归解决。一个树的最大深度等于其子树最大深度加一。

1. **基本情况 (Base Case):** 如果当前节点 `root` 为 `null`, 表示到达了空节点, 其深度为 0。
2. **递归步骤 (Recursive Step):**
 - 递归计算左子树的最大深度: `left_depth = maxDepth(root.left)`。
 - 递归计算右子树的最大深度: `right_depth = maxDepth(root.right)`。
 - 当前树的最大深度等于左右子树深度的较大值, 再加上当前节点本身 (深度加 1): `return max(left_depth, right_depth) + 1`。

执行流程示例 (`[3,9,20,null,null,15,7]`): 1. `maxDepth(3)` 2. `-> maxDepth(9)` 3. `-> maxDepth(null)` `-> 返回 0` 4. `-> maxDepth(null)` `-> 返回 0` 5. `-> max(0, 0) + 1` `-> 返回 1` (节点 9 的深度) 6. `-> maxDepth(20)` 7. `-> maxDepth(15)` `-> 返回 1` (类似节点 9 的计算) 8. `-> maxDepth(7)` `-> 返回 1` (类似节点 9 的计算) 9. `-> max(1, 1) + 1` `-> 返回 2` (节点 20 的深度) 10. `max(1, 2) + 1` `-> 返回 3` (节点 3 的深度)

方法二: 迭代 (广度优先搜索 - BFS / 层序遍历) 我们也可以使用层序遍历的方式来计算深度。每遍历一层, 深度就加一。

1. **初始化:**
 - 如果 `root` 为 `null`, 返回 0。
 - 创建一个队列 `queue`, 并将根节点 `root` 入队。
 - 初始化深度 `depth = 0`。
2. **层序遍历:**
 - 当队列不为空时, 进行循环:
 - 获取当前层的节点数量 `level_size = queue.size()`。
 - **深度加一:** `depth += 1`。
 - 循环 `level_size` 次, 处理当前层的所有节点:
 - * 将队首节点 `node` 出队。
 - * 如果 `node` 的左子节点不为空, 将其入队。
 - * 如果 `node` 的右子节点不为空, 将其入队。
3. **返回结果:** 循环结束后, `depth` 即为树的最大深度。

执行流程示例 (`[3,9,20,null,null,15,7]`):

步骤	queue	level_size	depth	操作
1	[3]	-	0	初始化
2	[3]	1	1	depth=1, 处理第 1 层
3	[9, 20]	1	1	出队 3, 入队 9, 20
4	[9, 20]	2	2	depth=2, 处理第 2 层
5	[20]	2	2	出队 9 (无子节点)
6	[15, 7]	2	2	出队 20, 入队 15, 7
7	[15, 7]	2	3	depth=3, 处理第 3 层
8	[7]	2	3	出队 15 (无子节点)
9	[]	2	3	出队 7 (无子节点)
10	[]	-	3	队列为空, 循环结束, 返回 depth=3

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import Optional
import collections

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class Solution:
    # 方法一: 递归 (DFS)
    def maxDepthRecursive(self, root: Optional[TreeNode]) -> int:
        """
        使用递归 (DFS) 计算二叉树的最大深度
        """
        # 基本情况: 空节点深度为 0
        if not root:
            return 0
        # 递归计算左子树深度
        left_depth = self.maxDepthRecursive(root.left)
        # 递归计算右子树深度
        right_depth = self.maxDepthRecursive(root.right)
        # 当前树的深度 = max(左子树深度, 右子树深度) + 1 (当前节点)
        return max(left_depth, right_depth) + 1

    # 方法二: 迭代 (BFS)
    def maxDepthIterative(self, root: Optional[TreeNode]) -> int:
        """
        使用迭代 (BFS / 层序遍历) 计算二叉树的最大深度
        """
        if not root:
            return 0

        queue = collections.deque([root])
        depth = 0
        while queue:
            # 当前层的节点数
```

```

    level_size = len(queue)
    # 处理完一层, 深度加 1
    depth += 1
    # 遍历当前层的所有节点
    for _ in range(level_size):
        node = queue.popleft()
        # 将下一层的节点入队
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
    return depth

# 最终提交可以选择其中一种方法
def maxDepth(self, root: Optional[TreeNode]) -> int:
    # return self.maxDepthRecursive(root)
    return self.maxDepthIterative(root)

```

4. 复杂度分析

- **递归方法 (DFS):**
 - 时间复杂度: $O(N)$, 其中 N 是树中的节点数。每个节点被访问一次。
 - 空间复杂度: $O(H)$, 其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下 (链状树) 为 $O(N)$, 平均情况下 (平衡树) 为 $O(\log N)$ 。
- **迭代方法 (BFS):**
 - 时间复杂度: $O(N)$ 。每个节点入队出队各一次。
 - 空间复杂度: $O(W)$, 其中 W 是树的最大宽度。在最坏情况下 (完全二叉树的最后一层) 可能达到 $O(N)$ 。

5. 如何在面试中讲解

1. **明确定义:**
 - “首先明确最大深度的定义: 从根节点到最远叶子节点的路径上的节点数。空树深度为 0。”
2. **递归思路 (DFS):**
 - “树的递归定义天然适合解决深度问题。一个树的深度是其子树最大深度加 1。”
 - “讲解递归三部曲: **终止条件** (节点为空, 返回 0), **递归左右子树**, **合并结果** ($\max(\text{左深度}, \text{右深度}) + 1$)。”
 - “分析复杂度: 时间 $O(N)$, 空间 $O(H)$ 。”
3. **迭代思路 (BFS):**
 - “另一种思路是层序遍历。我们逐层遍历树, 每遍历一层, 深度就加 1。”
 - “使用队列实现层序遍历。记录当前层的节点数, 处理完一层后深度加 1, 并将下一层节点入队。”
 - “分析复杂度: 时间 $O(N)$, 空间 $O(W)$ 。”
4. **对比与选择:**
 - “递归代码通常更简洁。BFS 迭代可以避免递归深度过大的问题, 且空间复杂度与树

的宽度有关。”

- “对于求最大深度，两种方法都可行且常用。”

5. **边界条件：**

- “需要处理空树（root 为 null）的情况，返回 0。”